Study Notes: Days 36–42 — EF Core + SQL

These notes follow Month 2, Week 2 of the roadmap, which is **Week 6 overall**. EF Core behavior depends on its version and database provider; examples identify their assumptions. SQL Server syntax is labeled explicitly. The scenarios are illustrative and are not claims about Sai's work.

Day 36 — EF Core fundamentals

Month 2 — Full-Stack Depth, Week 2 — EF Core + SQL (Week 6 overall).

Goal: Explain how EF Core maps objects to relational data, executes queries, tracks changes, and persists them safely.

EF Core, entities, and the model

Entity Framework Core is an object-relational mapper: it maps .NET types and relationships to a database model and translates supported LINQ queries into provider-specific database queries. An **entity** is a modeled object with a key; a `DbSet<TEntity>` is an entry point for querying and working with entities of a given type. Mapping conventions discover common patterns such as an `Id` key, while data annotations and `OnModelCreating` configure details such as relationships, constraints, lengths, and indexes.

An ORM reduces repetitive mapping code but does not remove database behavior. Query translation, collation, precision, null semantics, indexes, locking, and transactions depend on the provider and database. Inspect generated SQL and test against the actual provider for important paths; an in-memory fake cannot establish that a real database query translates or respects relational constraints.

`DbContext`, queries, and changes

`DbContext` represents a unit of work and maintains EF's model and change tracker. A typical web application registers it as scoped and uses one context per request or another short unit of work. It is not thread-safe: do not run concurrent operations on the same instance. Dispose it after use, and do not let an unexecuted query outlive its context.

LINQ queries are generally constructed first and executed when enumerated or when a terminal method such as `ToListAsync` runs. Filter and project before materialization when those operations should run in the database. EF translates supported expressions; unsupported expressions in most query positions produce a translation exception rather than silently fetching an entire table for client filtering. Inspect `ToQueryString()` or EF logging, and parameterize values rather than constructing unsafe SQL strings.

Tracked entities move through states such as `Added`, `Unchanged`, `Modified`, and `Deleted`. `SaveChangesAsync` detects changes and sends database commands. With relational providers, changes in one `SaveChanges` call are normally wrapped in a transaction when supported. A successful save does not automatically make external API calls or messages part of that transaction. Use cancellation where supported, surface failures, and design concurrency handling for competing writers.

Schema evolution

EF Core migrations record model changes as schema operations that can be reviewed and applied to an existing database. Generate migrations from the intended model, review the generated SQL, plan data backfills and deployment order, and test upgrades and rollback or recovery. A migration is not automatically safe for a large production table.

`EnsureCreated` is convenient for disposable examples and certain simple scenarios; it bypasses the normal migrations workflow and should not be casually mixed with migrations for an evolving production database. Choose a schema-management strategy deliberately.

Code example

With the .NET 10 SDK, create a console project and add `Microsoft.EntityFrameworkCore.Sqlite` version `10.0.0`. This complete example uses an in-memory SQLite database whose connection remains open for the duration of the context:

```
using System;
using System.Linq;
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;

await using var connection = new SqliteConnection("Data Source=:memory:");
await connection.OpenAsync();
var options = new DbContextOptionsBuilder<CatalogDb>()
    .UseSqlite(connection)
    .Options;

await using var db = new CatalogDb(options);
await db.Database.EnsureCreatedAsync();
db.Products.Add(new Product { Name = "Notebook" });
await db.SaveChangesAsync();

var names = await db.Products
    .AsNoTracking()
    .Where(p => p.Name.StartsWith("Note"))
    .Select(p => p.Name)
    .ToListAsync();
Console.WriteLine(string.Join(", ", names)); // Notebook

public sealed class CatalogDb : DbContext
{
    public CatalogDb(DbContextOptions<CatalogDb> options) : base(options) { }
    public DbSet<Product> Products => Set<Product>();
}

public sealed class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
}
```

The final query projects strings, not entities, so there are no result entities to track even without `AsNoTracking`. The call makes the read-only intent visible but is not the reason this projection avoids entity tracking.

DSA problem: Reverse words in a string

Problem: Reverse the order of words in a string. Treat whitespace as a separator, remove leading/trailing whitespace, and join the result with a single space. Do not reverse the characters within each word.

Approach: Split on whitespace, reverse the array of words, and join with a single space.

Complexity: O(n) time and O(n) additional space for an input of `n` UTF-16 code units.

```
using System;

Console.WriteLine(ReverseWords("  hello   world  ")); // world hello
Console.WriteLine($"[{ReverseWords("   ")}]");       // []

static string ReverseWords(string text)
{
    string[] words = text.Split((char[]?)null,
StringSplitOptions.RemoveEmptyEntries);
    Array.Reverse(words);
    return string.Join(" ", words);
}
```

Edge cases: Empty or whitespace-only input produces an empty string; one word is unchanged; tabs and other whitespace separators are normalized to spaces. Punctuation stays attached to its word. This reverses word order, not letters or Unicode grapheme clusters.

Likely interview questions

1. What do `DbContext`, `DbSet<T>`, an entity, and the EF model each represent?
2. When is a LINQ query translated and executed?
3. Why is `DbContext` normally short-lived and why must it not be used concurrently?
4. What do EF entity states and `SaveChangesAsync` do?
5. What does one `SaveChanges` transaction not include?
6. How do migrations differ from `EnsureCreated`?
7. Why is testing only against an in-memory fake insufficient for SQL translation?
8. What exactly does the reverse-words solution do with whitespace and punctuation?

Common pitfalls / misconceptions

- Assuming LINQ always translates efficiently or identically across providers.
- Sharing a context across concurrent work or returning a query backed by a disposed context.
- Treating `EnsureCreated` as a production migration strategy.
- Reversing the characters in each word when only word order should change.

Cheat-sheet recap

- EF maps entities and translates supported queries; the database still determines important semantics.
- Keep contexts scoped to a unit of work; await operations and do not share a context concurrently.
- Review migrations and generated SQL; use provider-backed tests.
- Reverse words: split, reverse word order, join; specify whitespace semantics.

Day 37 — Tracking vs NoTracking

Month 2 — Full-Stack Depth, Week 2 — EF Core + SQL (Week 6 overall).

Goal: Choose tracking behavior based on update needs, identity resolution, result shape, and measured cost.

Tracking queries

A tracking query attaches returned entity instances to the context's change tracker. EF maintains identity resolution within the context: when it encounters a key already tracked, it returns the existing instance rather than constructing a second tracked entity for the same key. Changes to tracked properties can be detected and persisted by `SaveChanges`.

Tracking is convenient for a short read-modify-save operation, but snapshots, identity maps, and change detection consume memory and CPU. A long-lived context accumulates state and may return an already-tracked instance whose current values differ from the latest database state. Tracking is not a cache invalidation or concurrency-control strategy. Use a fresh context, reload explicitly, and configure concurrency tokens when the business operation needs those guarantees.

No-tracking queries

`AsNoTracking()` returns entities without attaching them for later change detection. It is useful for read-only results that do not need context-level identity resolution. Changing a detached result does not by itself cause `SaveChanges` to update the database. You can explicitly attach or update an entity, but do so carefully: blindly marking a client-supplied graph modified can overwrite fields, violate authorization boundaries, or bypass a concurrency check.

For result graphs containing repeated references to the same entity, ordinary no-tracking queries can materialize separate instances for the same key. `AsNoTrackingWithIdentityResolution()` uses a temporary identity-resolution tracker while materializing the result without attaching those entities to the context afterward. This has a cost; choose it when shared object identity is useful, not by default for every query.

Projections, performance, and correctness

A projection containing only scalars or DTO values generally has no entity instances to track. However, an anonymous object or DTO that **contains an entity** can still cause that entity to be tracked by default. `AsNoTracking` does not fix a missing index, an unbounded query, excessive columns, or N+1 behavior. Compare real workload allocations and latency before assuming it is the dominant optimization.

Updating a detached object requires clear decisions about which fields may change, how original concurrency values are supplied, and what happens if another writer updated the row. `DbUpdateConcurrencyException` is a signal to handle a conflict, not a reason to silently overwrite the other writer.

Code example

The following fragment uses the `CatalogDb` and `Product` types from Day 36. Run it in a fresh context containing a product with `Id = 1`:

```
using Microsoft.EntityFrameworkCore;

var tracked = await db.Products.SingleAsync(p => p.Id == 1);
var detached = await db.Products.AsNoTracking().SingleAsync(p => p.Id == 1);

tracked.Name = "Updated by tracking";
detached.Name = "Not automatically saved";
await db.SaveChangesAsync();

Console.WriteLine(db.Entry(tracked).State);  // Unchanged after successful save
Console.WriteLine(db.Entry(detached).State); // Detached
```

The tracked instance's change is saved; modifying the no-tracking instance alone is not. A real update path should also enforce allowed fields and concurrency rules.

Likely interview questions

1. What does a tracking query do with a row whose key is already tracked?
2. What changes when `AsNoTracking` is used?
3. Why can a long-lived context show stale or unexpected entity values?
4. What does `AsNoTrackingWithIdentityResolution` add, and what does it cost?
5. Can a projection still track entities?
6. What risks arise from calling `Update` on a client-supplied object graph?
7. How do you handle competing updates without silently overwriting one writer?
8. Why is `AsNoTracking` not a substitute for checking SQL and indexes?

Common pitfalls / misconceptions

- Assuming a no-tracking entity is automatically updated by `SaveChanges`.
- Assuming every projection disables tracking, even when it contains an entity.
- Using one long-lived context as a reliable shared cache.
- Applying `Update` to untrusted disconnected graphs without field or concurrency checks.

Cheat-sheet recap

- Tracking supports identity resolution and read-modify-save.
- No tracking avoids attaching results; identity-resolution mode is a separate choice.
- Scalar-only DTO projections do not contain entities to track.
- Measure the whole query and define safe disconnected-update behavior.

Day 38 — Lazy vs eager vs explicit loading

Month 2 — Full-Stack Depth, Week 2 — EF Core + SQL (Week 6 overall).

Goal: Choose how related entities are loaded and identify N+1 queries, oversized joins, and context-lifetime hazards.

Eager loading

Eager loading requests related entities as part of the initial query, commonly with `Include` and `ThenInclude`. It is useful when the related data is known to be needed. Alternatively, a projection can request only the fields needed by the response, often avoiding a large tracked object graph.

Including multiple sibling collections can multiply rows in the SQL result—often called cartesian explosion—and repeat parent columns. `AsSplitQuery()` can avoid some multiplication by issuing multiple queries, but adds round trips and can observe inconsistent data between queries unless the chosen transaction and isolation level provide the required consistency. EF version, provider, ordering, and pagination details matter; inspect the generated SQL and measure.

Explicit loading

Explicit loading requests a navigation after an entity is already available, using `Entry(entity).Reference(...).LoadAsync()` or `Collection(...).LoadAsync()`. It is useful when the decision to load depends on a later condition. It makes the extra database call visible but can still produce N+1 queries if placed in a loop.

Typically the entity is tracked by the relevant context when using `Entry(...).LoadAsync`. Relationship fixup can populate navigations from entities already tracked in the same context, making a demonstration appear to have loaded more data than its query actually requested. Use fresh contexts when comparing strategies and inspect actual SQL.

Lazy loading

Lazy loading fetches related data when a navigation is accessed, if configured. EF Core does **not** enable proxy-based lazy loading merely because a navigation exists: proxies require the appropriate package and configuration, and proxyable entity types and navigations. EF also supports other lazy-loading mechanisms.

Lazy loading can make simple code convenient, but access inside a loop or serializer can trigger many unexpected database round trips. It depends on a usable context and can interact poorly with serialization and context lifetime. Do not assume one property access is free. Prefer explicit projections or deliberate eager/explicit loading when predictable query counts and API response shape matter.

Code examples

Assume `db.Orders` contains `Order` entities with an `Items` collection and use a **fresh context for each example**:

```
using Microsoft.EntityFrameworkCore;

// Eager loading: request the collection with the order query.
var orders = await db.Orders
    .Where(o => o.CustomerId == customerId)
    .Include(o => o.Items)
    .ToListAsync(cancellationToken);

// Explicit loading: fetch the collection later for one tracked order.
```

```
var order = await db.Orders.SingleAsync(o => o.Id == orderId, cancellationToken);
await db.Entry(order).Collection(o => o.Items).LoadAsync(cancellationToken);
```

These are alternative fragments, not necessarily operations to run one after the other on the same context. For proxy-based lazy loading, add the provider-compatible `Microsoft.EntityFrameworkCore.Proxies` package and configure it on the options builder:

```
// Configuration fragment; also configure the chosen database provider.
optionsBuilder.UseLazyLoadingProxies();

public class Order
{
    public int Id { get; set; }
    public virtual ICollection<OrderItem> Items { get; set; } = new
List<OrderItem>();
}

public class OrderItem
{
    public int Id { get; set; }
    public int OrderId { get; set; }
}
```

With proxies configured and a suitable live context, accessing `order.Items` may issue a query. The example omits the rest of the context and provider configuration; verify actual SQL and lifetimes in the application.

DSA problem: Valid palindrome

Problem: Determine whether text is a palindrome after ignoring everything except ASCII letters and digits and comparing letters case-insensitively.

Approach: Use two pointers from the ends, skip non-ASCII-alphanumeric characters, and compare the remaining characters after invariant lowercase conversion. Move inward until the pointers meet.

Complexity: O(n) time and O(1) additional space.

```
using System;

Console.WriteLine(IsPalindrome("A man, a plan, a canal: Panama")); // True
Console.WriteLine(IsPalindrome("race a car"));                     // False

static bool IsPalindrome(string text)
{
    int left = 0, right = text.Length - 1;
    while (left < right)
    {
        while (left < right && !IsAsciiAlphanumeric(text[left])) left++;
        while (left < right && !IsAsciiAlphanumeric(text[right])) right--;
        if (char.ToLowerInvariant(text[left]) !=
```

```
    char.ToLowerInvariant(text[right]))
                return false;
            left++;
            right--;
        }
        return true;
    }

    static bool IsAsciiAlphanumeric(char c) =>
        (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || (c >= '0' && c <= '9');
```

Edge cases: Empty or punctuation-only input is a palindrome under this definition; digits are significant. Non-ASCII letters are ignored here by design. For international text, define normalization, case folding, and whether comparison operates on Unicode scalar values or grapheme clusters before choosing an algorithm.

Likely interview questions

1. How do eager, explicit, and lazy loading differ?
2. What configuration is required for proxy-based lazy loading?
3. How can lazy or explicit loading create an N+1 problem?
4. Why can multiple collection includes multiply returned rows?
5. What are the round-trip and consistency tradeoffs of split queries?
6. How can relationship fixup confuse a loading demonstration?
7. When is a DTO projection preferable to loading a full entity graph?
8. What exact character rules does the palindrome algorithm implement?

Common pitfalls / misconceptions

- Assuming lazy loading is enabled automatically.
- Accessing navigations inside a loop without inspecting query count.
- Assuming split queries are always faster or always consistent.
- Calling an ASCII-only palindrome algorithm fully Unicode-aware.

Cheat-sheet recap

- Eager = request related data up front; explicit = request later deliberately; lazy = fetch on navigation access when configured.
- Measure query count, row multiplication, round trips, and context lifetime.
- Two pointers solve the specified ASCII palindrome in O(n) time and O(1) space.

Day 39 — Transactions

Month 2 — Full-Stack Depth, Week 2 — EF Core + SQL (Week 6 overall).

Goal: Define transaction boundaries and isolation, handle concurrent changes, and distinguish database atomicity from distributed reliability.

Atomicity and transaction boundaries

A database transaction groups operations so they commit or roll back as a unit, subject to the database's guarantees. ACID describes atomicity, consistency with enforced constraints, isolation, and durability. “Consistency” does not mean every business rule is automatically enforced: application logic, constraints, and isolation must work together.

With a relational provider that supports transactions, a single EF Core `SaveChanges` call is normally transactional. Use an explicit transaction when multiple saves or database commands must succeed together. Dispose an uncommitted transaction to roll it back; commit only after all required database work succeeds. Keep transactions short to limit lock duration and contention.

Isolation and concurrent updates

Isolation determines what concurrent transactions can observe and which anomalies are possible. Typical terms include dirty reads, non-repeatable reads, and phantoms, but exact behavior depends on the database and configuration. For example, SQL Server `READ COMMITTED` behavior differs depending on whether read-committed snapshot is enabled; snapshot and serializable isolation have different conflict and locking characteristics. Check the actual provider rather than memorizing a universal table of guarantees.

Transactions alone do not automatically prevent lost updates or enforce every cross-row invariant. Use appropriate isolation, atomic conditional updates, unique/check constraints, or optimistic concurrency tokens as the problem requires. EF Core can detect conflicts with configured concurrency tokens and throw `DbUpdateConcurrencyException`; resolve the conflict deliberately rather than blindly retrying with stale values.

Retries, savepoints, and external effects

Deadlocks and transient failures can make transactions fail; retry only when the operation is safe to repeat and the error is classified as transient. EF execution strategies and user-managed transactions must be coordinated so the **whole transaction** is retried as a unit. A connection failure during commit can leave the outcome unknown, so a retry may duplicate work unless a durable idempotency key or equivalent design resolves ambiguity.

EF may create a savepoint before `SaveChanges` inside an existing transaction where the provider supports it; provider and configuration limitations apply. Savepoints do not replace a complete recovery plan.

A database transaction does not atomically include an arbitrary HTTP call or message broker. A transactional outbox can persist a message alongside business changes, then deliver it separately; delivery may be repeated, so consumers should handle duplicates appropriately. Compensating actions are a separate business design, not an automatic rollback of external reality.

Code example

This is an EF Core application fragment for a relational provider that translates the decimal operations, such as SQL Server. Assume `BankDb.Accounts` has `Id` and `Balance` properties. It uses atomic conditional database updates inside one explicit transaction:

```
using Microsoft.EntityFrameworkCore;

static async Task TransferAsync(
```

```
    BankDb db, long sourceId, long destinationId, decimal amount,
    CancellationToken cancellationToken)
{
    if (sourceId == destinationId)
        throw new ArgumentException("Source and destination must differ.");
    if (amount <= 0)
        throw new ArgumentOutOfRangeException(nameof(amount));

    await using var transaction = await
db.Database.BeginTransactionAsync(cancellationToken);

    int debited = await db.Accounts
        .Where(a => a.Id == sourceId && a.Balance >= amount)
        .ExecuteUpdateAsync(
            setters => setters.SetProperty(a => a.Balance, a => a.Balance -
amount),
            cancellationToken);
    if (debited != 1)
        throw new InvalidOperationException("Source is missing or has insufficient
funds.");

    int credited = await db.Accounts
        .Where(a => a.Id == destinationId)
        .ExecuteUpdateAsync(
            setters => setters.SetProperty(a => a.Balance, a => a.Balance +
amount),
            cancellationToken);
    if (credited != 1)
        throw new InvalidOperationException("Destination is missing.");

    await transaction.CommitAsync(cancellationToken);
}
```

If the destination update fails, disposing the uncommitted transaction rolls back the debit. `ExecuteUpdateAsync` executes immediately and bypasses the change tracker, so any already-tracked account objects may be stale. This fragment is not a complete banking transfer: it lacks currency and precision rules, authorization, a durable ledger, idempotency, and a retry/unknown-commit design. Opposite-direction concurrent transfers can deadlock; handle that deliberately rather than assuming this code eliminates deadlocks.

Likely interview questions

1. What is atomic about a database transaction, and what does it not cover?
2. When is an explicit transaction needed around EF Core operations?
3. How can a transaction still allow a lost update or an invariant violation?
4. How do isolation levels and provider configuration affect reads and locks?
5. What does a concurrency token detect, and how should a conflict be handled?
6. Why must a retry strategy coordinate the entire transaction?
7. What makes a commit outcome ambiguous after a connection failure?
8. How does an outbox help coordinate database changes and message publication?

Common pitfalls / misconceptions

- Assuming one transaction automatically enforces all business rules or includes external calls.
- Retrying a non-idempotent transaction without considering an unknown commit outcome.
- Holding locks while waiting on remote services.
- Assuming `ExecuteUpdate` synchronizes already-tracked entities.

Cheat-sheet recap

- One supported relational `SaveChanges` call is normally atomic; use explicit boundaries for larger units of work.
- Isolation, constraints, conditional updates, and concurrency tokens solve different problems.
- Coordinate retries and account for unknown commit outcomes.
- Use an explicit reliability pattern for database-plus-message workflows.

Day 40 — SQL indexing

Month 2 — Full-Stack Depth, Week 2 — EF Core + SQL (Week 6 overall).

Goal: Explain how indexes support access paths and ordering, design a candidate index for a query, and measure its read/write tradeoffs.

How indexes work

An index is an additional data structure that helps a database locate or order rows without always reading the entire table. Common relational indexes use B-tree structures, with ordered keys and links to the underlying data. Exact storage and access mechanics vary by database and index type. An index can reduce reads for selective predicates or support an `ORDER BY`, but it consumes storage and adds work to inserts, updates, and deletes.

A seek is not automatically superior to a scan. A scan can be appropriate for a small table, a low-selectivity predicate, or a query returning much of the table. The optimizer chooses a plan based on estimated costs and available statistics; confirm with the actual plan, reads, timing, and workload.

Composite and covering indexes

For a composite B-tree index, key order matters. Equality predicates on leading keys often help narrow a range; subsequent range and ordering columns may support the query, depending on the exact predicate, sort direction, and optimizer. There is no universal column-order recipe: consider selectivity, the whole query, reuse across the workload, and write cost.

An index can be **covering** for a query if it supplies all required columns without extra lookups to the base table or clustered index. SQL Server supports `INCLUDE` columns that are stored in the nonclustered index but are not part of the sort key. Other databases offer different mechanisms. Covering a wide query can make an index expensive, so avoid including columns indiscriminately.

Sargability, statistics, and operations

Predicates that allow the optimizer to use an index effectively are often called sargable. Applying a function to an indexed column or introducing an implicit type conversion may prevent an efficient seek, although actual

behavior depends on the optimizer. For a date interval, prefer a half-open range such as `CreatedAt >= @start AND CreatedAt < @end` when it expresses the requirement; ensure parameter types match column types.

Statistics affect cardinality estimates and plan selection. Data growth or skew can make an old plan unsuitable. Index maintenance, statistics updates, and online creation capabilities vary by provider and deployment; measure impact on write throughput, blocking, storage, and maintenance windows. A unique index can also enforce a uniqueness invariant, but nullable-key behavior and filtering depend on the database.

Code example

This is **SQL Server** syntax. Assume `dbo.Orders` has `TenantId`, `Status`, `CreatedAt`, `Id`, and `Total` columns:

```
CREATE INDEX IX_Orders_Tenant_Status_Created_Id
ON dbo.Orders (TenantId, Status, CreatedAt DESC, Id DESC)
INCLUDE (Total);

DECLARE @tenantId int = 42;
DECLARE @status nvarchar(20) = N'Open';
DECLARE @start datetime2 = '2026-01-01T00:00:00';
DECLARE @end datetime2 = '2026-02-01T00:00:00';

SELECT TOP (50) Id, CreatedAt, Total
FROM dbo.Orders
WHERE TenantId = @tenantId
  AND Status = @status
  AND CreatedAt >= @start AND CreatedAt < @end
ORDER BY CreatedAt DESC, Id DESC;
```

This is a candidate index for the shown filter, projection, and ordering—not a promise that the optimizer will choose it or that it benefits every workload. Match parameter types to the actual schema, inspect the plan, and evaluate write overhead before adding it.

DSA problem: Longest substring without repeating characters

Problem: Return the length of the longest contiguous substring containing no repeated UTF-16 code units. For example, `abcabcbb` returns `3`.

Approach: Maintain a sliding window and a dictionary of the most recent index for each code unit. If the current code unit last appeared inside the window, move the window start just after that occurrence. Record the current index and update the best length.

Complexity: Expected O(n) time and O(k) space for `k` distinct code units encountered.

```
using System;
using System.Collections.Generic;

Console.WriteLine(LongestUniqueSubstring("abcabcbb")); // 3
Console.WriteLine(LongestUniqueSubstring("abba"));     // 2
```

```
Console.WriteLine(LongestUniqueSubstring(""));         // 0

static int LongestUniqueSubstring(string text)
{
    var lastSeen = new Dictionary<char, int>();
    int start = 0, best = 0;
    for (int end = 0; end < text.Length; end++)
    {
        char current = text[end];
        if (lastSeen.TryGetValue(current, out int previous) && previous >= start)
            start = previous + 1;
        lastSeen[current] = end;
        best = Math.Max(best, end - start + 1);
    }
    return best;
}
```

Edge cases: Empty input returns zero; repeated adjacent characters work; `abba` demonstrates that the start pointer must never move backward. This returns a length in UTF-16 code units. Unicode scalar values or grapheme clusters require a different iteration and length definition.

Likely interview questions

1. What does a B-tree index help the database avoid, and what does it cost?
2. Why can a scan be the right plan?
3. How does composite index key order relate to equality, range, and sorting?
4. What is a covering index, and what are the tradeoffs of included columns?
5. How can a function or implicit conversion affect index use?
6. Why can stale or unrepresentative statistics change the selected plan?
7. How would you measure an index's impact on both reads and writes?
8. Why must the sliding-window start never move backward?

Common pitfalls / misconceptions

- Adding an index to every column without accounting for write and storage costs.
- Treating a seek as inherently good and a scan as inherently bad.
- Designing a composite index from one predicate without considering ordering and projection.
- Treating UTF-16 code units as universally equivalent to user-perceived characters.

Cheat-sheet recap

- Indexes trade write/storage cost for potentially cheaper reads and ordering.
- Key order, selectivity, query shape, statistics, and coverage affect usefulness.
- Validate the actual plan and workload; don't assume an index will be chosen.
- Longest unique substring: track last-seen indices and move the window start only forward.

Day 41 — Execution plans

Month 2 — Full-Stack Depth, Week 2 — EF Core + SQL (Week 6 overall).

Goal: Read a plan as evidence of how a query executes, compare estimates with actual behavior, and validate changes with measurements.

What a plan shows

A query optimizer selects an execution plan: a tree of operators describing access paths, joins, sorting, aggregation, and other work. An **estimated plan** shows the optimizer's predicted strategy and cardinalities without executing the query. An **actual plan** includes runtime information from an execution, such as actual row counts and warnings where the database exposes them.

Read from data access through joins and later operators to the final result, but use the tool's operator conventions rather than assuming every graphical plan draws flow in the same direction. Compare estimated and actual rows at important operators. Large discrepancies can lead to poor join choices, inadequate memory grants, spills, or excessive work. They are a clue to investigate statistics, skew, predicates, parameter values, and correlations—not proof of one particular cause.

Common operators and diagnosis

An index seek narrows access by key; a scan reads a broader range. A key lookup retrieves additional columns not covered by an index and can become expensive when repeated for many rows. Nested-loops, hash, and merge joins have different cost profiles depending on input sizes, ordering, and available indexes. Sorts and hash operations can request memory and may spill to disk if the grant is insufficient.

Do not optimize based solely on a graphical cost percentage: estimates and costs are model-dependent. Correlate the plan with actual duration, logical and physical reads, row counts, waits, CPU, and concurrency. Check for implicit conversions, non-sargable predicates, unexpected sorts, missing or excessive rows, and parameter-sensitive plans. A missing-index suggestion is a hypothesis; it may overlap existing indexes or harm write performance.

Safe measurement

In SQL Server, an actual execution plan requires executing the query; `SET STATISTICS IO, TIME ON` reports reads and timing for the session. Use representative parameters and data distributions, compare warm and cold-cache behavior where relevant, and avoid disrupting production. Query plans and logs may contain sensitive data or parameter values.

Other engines differ: PostgreSQL `EXPLAIN ANALYZE` executes the statement, including side effects for a data-changing statement, while SQLite `EXPLAIN QUERY PLAN` reports a higher-level strategy. Use provider-specific tools and understand their operational impact.

Code example

This is a **SQL Server** diagnostic query. Enable “Include Actual Execution Plan” in the SQL client before running it. It assumes the same `dbo.Orders` schema as Day 40:

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

DECLARE @tenantId int = 42;
```

```
DECLARE @status nvarchar(20) = N'Open';
DECLARE @start datetime2 = '2026-01-01T00:00:00';
DECLARE @end datetime2 = '2026-02-01T00:00:00';

SELECT TOP (50) Id, CreatedAt, Total
FROM dbo.Orders
WHERE TenantId = @tenantId
  AND Status = @status
  AND CreatedAt >= @start AND CreatedAt < @end
ORDER BY CreatedAt DESC, Id DESC;

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
```

Record logical reads, elapsed time, actual versus estimated rows, and relevant operators before and after a proposed change. Match parameter types to the real schema; a test using only one tenant may miss skew or parameter-sensitive behavior.

Likely interview questions

1. How do estimated and actual execution plans differ?
2. What can a large mismatch between estimated and actual rows indicate?
3. When can repeated key lookups become expensive?
4. What do sorts, hash joins, memory grants, and spills tell you?
5. Why should a scan or an optimizer cost percentage not be judged in isolation?
6. How can parameter values and data skew produce different plan needs?
7. What evidence would you collect before accepting a missing-index suggestion?
8. What operational risk does `EXPLAIN ANALYZE` pose for a data-changing statement?

Common pitfalls / misconceptions

- Treating the estimated plan as proof of actual runtime behavior.
- Tuning one operator without measuring total query and workload impact.
- Applying every missing-index suggestion without checking overlap or writes.
- Running an executing diagnostic statement against production data without understanding side effects.

Cheat-sheet recap

- Compare estimates with actual rows and runtime metrics.
- Investigate access paths, joins, lookups, sorts, spills, conversions, and parameter sensitivity.
- Test representative data and measure before and after; use provider-specific diagnostics safely.

Day 42 — Performance troubleshooting: “API slowed after DB grew from 100K to 10M rows — how do you investigate?”

Month 2 — Full-Stack Depth, Week 2 — EF Core + SQL (Week 6 overall).

Goal: Diagnose a scale-related slowdown with evidence, choose a targeted fix, and verify both performance and correctness.

Establish the symptom and baseline

First define what “slowed” means: affected endpoint and operations, p50/p95/p99 latency, throughput, error and timeout rates, concurrency, and when the regression began. Compare the same workload, data distribution, deployment, and cache conditions before and after where possible. Identify whether the problem affects all tenants or only a skewed subset, and whether the slowdown is in the application, network, database, or a downstream service.

Trace a representative request across boundaries. Record database query count, duration, rows read and returned, connection-pool wait, application CPU and allocations, thread-pool behavior, and dependency latency. Avoid logging sensitive parameters. A database that grew from 100K to 10M rows is a useful clue, not proof that the table size alone is the root cause.

Inspect the query and plan

Capture the actual SQL generated by EF, parameter types, representative parameter values or safe classifications, query count, and actual execution plan. Look for an unbounded result, filtering after `AsEnumerable` or `ToList`, N+1 navigation loading, unnecessary entity graphs, expensive sorts, scans or lookups with high reads, implicit conversions, inaccurate cardinality estimates, lock waits, and parameter-sensitive plans. Check statistics and existing indexes before proposing another one.

Measure database resource pressure and contention as well: CPU, memory, I/O latency, blocking, deadlocks, transaction duration, and connection saturation. A plan that is fast in isolation can behave differently under concurrent load. Check if a recent application, schema, data distribution, or configuration change coincided with the regression.

Select and validate a fix

Choose the fix that matches evidence: move translatable filters to the database, project only needed fields, eliminate N+1 queries, use deterministic bounded pagination, adjust an index or query shape, update statistics where appropriate, shorten transactions, or reduce unnecessary work. Caching or scaling may help a measured bottleneck but adds cost and consistency concerns; do not use either to hide an unbounded query without understanding it.

Compare before and after using production-like volume, representative tenants and parameters, concurrency, and warm/cold cache conditions where relevant. Report p95/p99, query count, logical reads, resource usage, error rate, and write overhead, alongside a correctness check for result set, ordering, authorization, and consistency. Define a rollback or feature rollout plan and monitor after deployment. An improvement is not established by one fast local run.

Code example

This EF Core application fragment assumes an `Orders` set with `TenantId`, `Status`, `CreatedAt`, `Id`, and `Total`. It requests a bounded, deterministic DTO projection and passes cancellation to the database operation:

```
using Microsoft.EntityFrameworkCore;

int limit = 50;
var recentOrders = await db.Orders
```

```
        .Where(o => o.TenantId == tenantId && o.Status == "Open")
        .OrderByDescending(o => o.CreatedAt)
        .ThenByDescending(o => o.Id)
        .Select(o => new OrderSummary(o.Id, o.CreatedAt, o.Total))
        .Take(limit)
        .ToListAsync(cancellationToken);

    public sealed record OrderSummary(long Id, DateTime CreatedAt, decimal Total);
```

This is a candidate query, not proof of an optimization: inspect its SQL and plan, ensure the tenant filter comes from an authorized context, and evaluate an index against the real workload. Since the projection contains only scalar values, it does not materialize tracked `Order` entities. A follow-up page needs a defined cursor or other pagination contract.

Real-world tie-in: a scale-related slowdown

The roadmap asks whether you actually encountered a scale-related slowdown at **Infor or NCR**. If you did, identify the real workload, symptom, evidence, bottleneck, your contribution, fix, and measured result. If you did not, say so and discuss the hypothetical investigation above instead. Do not invent a production incident, metrics, or ownership.

Model answer shape: “In [actual system and workload], we observed [measured symptom] under [conditions]. We used [actual evidence] and found [verified bottleneck]. I contributed [your specific work]. We chose [actual change] over [alternative] because [tradeoff]. We verified [real measurements and correctness checks]. The remaining risk was [honest limitation].”

Likely interview questions

1. What measurements would you collect before assuming the database is the bottleneck?
2. How can you identify N+1 queries or client-side filtering in an EF endpoint?
3. What would you compare in the actual execution plan at 100K versus 10M rows?
4. How do data skew and parameter-sensitive plans affect the investigation?
5. When would pagination, projection, indexing, or caching be appropriate?
6. How do you evaluate an index's effect on writes and other queries?
7. What correctness and security checks accompany a query optimization?
8. Describe a real scale issue if you experienced one; otherwise explain a hypothetical investigation honestly.

Common pitfalls / misconceptions

- Adding an index, cache, or more hardware before establishing the bottleneck.
- Measuring one query in isolation while ignoring concurrency, waits, or tenant skew.
- Improving latency by accidentally changing results, ordering, authorization, or consistency.
- Inventing an Infor/NCR incident or claiming an unmeasured improvement.

Cheat-sheet recap

- Baseline the symptom and trace the request before choosing a fix.
- Inspect generated SQL, query count, actual plans, rows, waits, and resource usage.

- Fix the measured cause; validate performance, correctness, and write impact under representative load.
- For a real story: symptom → evidence → cause → contribution → verification → tradeoff.